PROJECT FORENSICS REPORT
LOGICIEL DE GESTION FINANCIÈRE

AI ENGINEERING WORKFLOW EXTRACTION V0

Objet
Reconstruction forensique du workflow réellement pratiqué : conception, IA, responsabilités, décisions, gates de qualité, erreurs, évolution et règles réutilisables.

Élément	Valeur
Statut	Forensic extraction
Version	V0
Périmètre	Projet complet observé jusqu'à l'intégration V1
Méthode	Reconstituer d'abord, évaluer ensuite

Légende : Confirmé = observé explicitement ; Inféré = conclusion tirée de plusieurs éléments ; Recommandation = règle proposée.

1. Project identity

Dimension	Constat
Nom	Logiciel de gestion financière
Contexte client	EL-BARKA EMBALLAGE
Type	Application desktop Windows hors ligne
Domaine	Gestion financière opérationnelle : ventes, achats, dépenses, paiements, trésorerie, créances/dettes, reporting
Stack	C# / .NET 10 LTS / WPF / MVVM / EF Core / SQLite / xUnit / logging local / Inno Setup
Architecture	Modular monolith
Équipe	2 développeurs humains + agents IA spécialisés
Données	SQLite local ; montants en long / unités mineures ; INTEGER côté SQLite
État observé	Recovery V1 + Dashboard V2/Reports intégrés sur origin/integration ; release/v1.0-rc distincte

Invariant financier
Les ventes/achats créent des obligations mais ne déplacent pas directement la trésorerie ; les paiements et transferts la déplacent. Les créances/dettes sont dérivées des obligations moins allocations.

2. Chronological reconstruction

Étape	Reconstruction
1 — Discovery	Compréhension du métier, du périmètre et des opérations attendues.
2 — Formalisation	CDC, modélisation fonctionnelle, modèle de données, UX/UI et architecture deviennent des références validées.
3 — Foundation	C#/.NET/WPF/EF Core/SQLite, migrations, RBAC, opérations financières et invariants.
4 — Integrity hardening	Audits adversariaux ciblés : allocation, stockage SQLite et contrôles associés.
5 — UAT	Initialisation fraîche, soldes d'ouverture, ventes/achats/paiements/dépenses/transferts/RBAC.
6 — Completeness audit	UAT révèle des gaps : backup/restore UI, pré-migration backup, salaire avec employé, backup quotidien.
7 — Recovery V1	Chantier séparé : implémentation, audit, corrections, tests, acceptation, commit/push.
8 — Client feedback	Demandes : dashboard visuel, créances/dettes, paiements obligation-centric, ticket, verrouillage manuel.
9 — Dashboard / Reports	Dashboard devient opérationnel ; comparaison/évolution déplacées dans Rapports.
10 — Runtime / visual QA	Données réalistes révèlent crash DueAlerts ; puis correction des overlays/visibilités des graphes et polish UI.
11 — Freeze	Branche Dashboard V2 validée avec tests/build/migrations/visual UAT puis figée.
12 — Integration	Recovery puis Dashboard/Reports fusionnés localement sur integration, revalidés et poussés.
13 — État actuel	Integration avancée ; smoke runtime post-intégration explicitement séparé de la validation déjà effectuée.

3. Actual engineering workflow

Le workflow réellement observé est un **processus itératif à gates** : cadrage documentaire → conception → implémentation bornée → tests → audit → validation humaine → Git → intégration.

Phase	Travail	Gate
Discovery	Compréhension métier	Périmètre suffisamment stable
Specification	CDC/règles/modèles	Version validée
Design	Data/UX/architecture	Design accepté
Implementation	Agent code + tests	Tests/build/revue
Adversarial review	Recherche de failles/cas limites	PASS / corrections
Manual UAT	Parcours réel / visuel	Acceptation humaine
Git freeze	Commit/push contrôlé	SHA + refs vérifiés

Phase	Travail	Gate
Integration	Merge local + régression	Integration validée

Règle structurante
« Compile » ne signifie pas « terminé ». La fermeture exige des preuves techniques et, pour l'UX/runtime, une validation humaine.

4. AI use

Usage	Fonction
Lead conversationnel	Analyse, découpage, choix du chantier, critères d'acceptation, revue des résultats.
Codex	Implémentation, tests, diagnostics et rapports techniques sur périmètre borné.
Astra	Audit architecture/sécurité/migrations/cas adversariaux.
Sol	Implémentations substantielles WPF/features/logic métier.
Luna	Tâches bornées : Git, préparation, polish, freeze/UAT.
Analyse screenshots	Revue détaillée de l'UX/UI et des graphes.
Harness de données	Seed temporaire synthétique pour rendre les états visuels observables.
Validation automatisée	Tests, build, migrations, drift, vulnérabilités, invariants.

Le modèle d'usage est **agentique mais supervisé** : l'IA exécute beaucoup de travail, mais le produit, le périmètre et les gates restent gouvernés humainement.

5. Human vs AI responsibility

Responsabilité	Humain	IA
Décision métier	Décide	Analyse/formalise
Périmètre V1	Tranche	Alerte/propose
Architecture	Valide	Conçoit/challenge
Code	Supervise/accepte	Implémente
Tests	Définit attentes	Automatise/exécute
Audit	Déclenche/interprète	Exécute
UAT visuel	Décision finale	Prépare/diagnostique
Git/release	Contrôle final	Exécute sur instruction

Décision finale
Les choix produit — notamment Dashboard opérationnel vs Rapports analytiques, l'acceptation de la visibilité cross-user et le report des suppressions financières risquées — restent des décisions humaines.

6. Decision-making

- **Client → produit :** les retours client ont modifié le backlog et la UX.
- **Architecture :** séparation Dashboard/Rapports, modular monolith, obligations + allocations, minor units.
- **Risque :** ne pas implémenter une modification/suppression financière complexe sans invariants explicites.
- **Sécurité :** RBAC, zones responsables-only, backups, protection des migrations.
- **Environnement :** l'erreur SQLite readonly a été traitée comme problème d'ACL, pas comme bug métier.
- **Release :** pas de commit/push avant ACCEPT ; merge contrôlé ; pas de force push.

7. Validation & quality gates

Gate	Preuve observée
Tests ciblés	Recovery/storage 254 PASS ; Dashboard/Reports ; WPF ciblés ; tests de binding.
Full suite	1109/1109, puis 1111/1111, puis 1130/1130 après intégration selon le périmètre.

Gate	Preuve observée
Build	0 warnings / 0 errors sur les validations rapportées.
Foundation	PASS.
Migrations	001–004 uniquement ; drift contrôlé.
Vulnérabilités	Scans rapportés sans vulnérabilités bloquantes.
Runtime	Smoke ciblé login/navigation ; smoke post-merge encore séparé.
Visual UAT	Acceptation humaine après corrections UI/graphes.
Git	SHA, branche, remote et working tree vérifiés.
Data safety	Backups/fingerprints et répertoires protégés pendant les opérations sensibles.

Philosophie
Le projet a convergé vers une logique de **preuve avant promotion** : chiffres, reproductions, tests négatifs et vérifications de refs plutôt qu'une simple affirmation.

8. Mistakes / false starts / corrections

Problème	Cause	Correction
Dashboard navigation	Initialisation avant création de commande	Réordonnancement + tests navigation.
Graphes invisibles	Converter bool/Visibility inadéquat	Converter target-aware + visibilité mutuellement exclusive.
DueAlerts crash	Binding TwoWay sur propriétés read-only	OneWay + régression + negative control.
SQLite readonly	ACL de fichiers/WAL/SHM	ACL ciblées Data/Backups.
Completeness gaps	UAT a révélé fonctions absentes	Recovery V1 + backlog priorisé.
Daily backup retry	Nom de fichier invalide pouvait supprimer le retry	Correction du nommage/logique.
Backup service nullable	Protection théorique contournable	Service obligatoire + échec avant migration.
Dashboard trop analytique	Comparaison placée initialement au mauvais endroit	Comparaison/évolution → Rapports.
Premier polish UI	Tabs natifs/cadres/spacing/controls	Cycle visual QA ciblé.
Intégration partielle	Timeout NuGet SkiaSharp/OpenTK	Revalidation séparée puis 1130 PASS.

9. Evolution during project

L'évolution majeure concerne le **niveau de contrôle de l'ingénierie assistée par IA**.

Avant	Évolution	État mature
Demandes larges	Tâches bornées	Prompts contractuels
Confiance agent	Vérification progressive	Confiance conditionnée par preuves
Tests	Fonctionnalité	Tests ciblés + full + négatifs
Git	Outil d'exécution	Gate de promotion
UAT	Parcours manuels	Données réalistes + visual QA
Sécurité	RBAC	RBAC + recovery + audits + environnement
Scope	Backlog large	Priorité client + report du risque
Data	DB utile au test	Protection + backups + harness isolé

10. Reusable engineering rules

- Une tâche IA doit avoir contexte, périmètre et interdits explicites.
- Pas de Git pendant l'implémentation sauf tâche Git-only explicite.
- Un changement financier doit être vérifié contre les invariants métier.
- Pour chaque migration : Up, Down, stockage réel, drift et effets métier.

- Tout bug critique : reproduction → cause racine → correction → régression.
- Tester WPF avec données non vides et états limites, pas uniquement avec une DB vide.
- Différencier bug applicatif, données et environnement.
- Les fixtures de démonstration doivent être isolées et réversibles.
- Validation visuelle humaine pour les changements UX à fort impact.
- Avant merge : refs, merge-base, fichiers, tests ; après merge : régression.
- Une déviation à une règle documentaire doit être explicitement enregistrée.
- Les modifications/suppressions financières complexes exigent un modèle d'invariants.
- Préférer les états dérivés quand cela évite les soldes redondants.
- Un résultat chiffré est préférable à un simple « OK ».

11. Anti-patterns

Anti-pattern	Risque
Coder avant de verrouiller les règles	Fonctionnalité correcte techniquement mais incorrecte métier.
Laisser l'agent choisir le périmètre	Scope creep / changements non demandés.
Tester uniquement unitaires	Bugs WPF/runtime/environment non couverts.
Tester uniquement DB vide	Défauts liés aux données réelles non déclenchés.
Pousser directement sur integration	Isolation et rejet plus difficiles.
Broad ACL fix	Surface de sécurité inutilement large.
Build vert = release ready	UAT, runtime, migrations et sécurité restent à vérifier.
Tout faire entrer en V1	Risque disproportionné sur les opérations financières.
Docs non traçables	Source de vérité difficile à synchroniser avec le dépôt.

12. Prompt engineering profile

Le prompting a évolué vers un modèle **contractuel**.

Composant	Pattern
Context	Projet, branche, baseline, rôle agent.
Goal	Une tâche précisément bornée.
Scope	Fichiers/zones concernés.
Constraints	Pas de migration/DB/Git/refactor hors scope selon le chantier.
Validation	Tests, build, migrations, sécurité.
Evidence	Fichiers, chiffres, cause racine, risques.
Stop	Attendre ACCEPT avant promotion.

Pattern réutilisable
Goal → **Context** → **Scope** → **Constraints** → **Implementation** → **Validation** → **Evidence** → **Stop**.

13. Tooling / execution model

Outil	Rôle
Git/GitHub	Branches, SHAs, freezes, integration.
Agents IA	Code, diagnostic, tests, rapports.
Astra	Audit adversarial.
dotnet/xUnit	Build et tests.
EF Core/SQLite	Persistance, migrations, intégrité.
WPF/LiveCharts	Desktop et visualisation.

Outil	Rôle
PowerShell/terminal	Exécution reproductible.
ProgramData	Données locales UAT/production.
Harness	Seed synthétique temporaire.
Screenshots	QA visuelle.

14. Project management model

Le pilotage ressemble à un **Kanban à gates** plutôt qu'à des sprints temporels stricts.

- **Lead central :** mémoire et coordination du projet.
- **Parallelisation :** DEV1/DEV2 si surfaces indépendantes.
- **Single migration owner :** un seul porteur de migrations à la fois.
- **Frozen baselines :** points de comparaison stables.
- **Client-priority reprioritization :** les demandes réordonnent le backlog.
- **Risk-based deferral :** reporter les zones à forte complexité de sûreté.
- **Evidence-driven closure :** fermeture par preuves et acceptation.

15. Future AI engineering agent capabilities

- Lire les documents validés et détecter automatiquement les divergences avec le code.
- Construire le graphe exigences → règles → données → services → UI → tests.
- Refuser les changements hors périmètre.
- Détecter les violations possibles d'allocation, avance et trésorerie.
- Simuler annulations, surpaiements, paiements multiples, dates limites et permissions.
- Exécuter des tests UI avec états vides et non vides.
- Comparer screenshots avant/après et détecter régressions de layout.
- Vérifier migrations Up/Down et schéma SQLite.
- Générer un dossier de release reproductible avec hashes et provenance.
- Détecter ACL, chemins ProgramData et dépendances environnementales.
- Maintenir un registre ADR de décisions.
- Produire une confiance fondée sur preuves sans remplacer l'acceptation humaine.

16. Knowledge buckets

Bucket	Contenu
A — Business	Ventes, achats, dépenses, paiements, obligations, avances, créances/dettes, trésorerie.
B — Product	Modules, navigation, Dashboard, Rapports, Administration, RBAC.
C — Data	SQLite, EF Core, migrations, minor units, invariants.
D — UX/UI	WPF, MVVM, design system, dashboards, listes, timelines.
E — Security	RBAC, backup/restore, migrations, lock, session policy.
F — Quality	Tests, négatifs, UAT, visual QA, regression.
G — Delivery	Branches, freezes, merge, release, installation.
H — AI orchestration	Rôles modèles, prompts, gates, délégation.
I — Client decisions	Priorités, acceptations, déviations.

17. Knowledge not to enter the agent

Exclusions
Mots de passe, secrets, tokens, clés privées, données financières réelles inutiles, copies de DB sensibles et informations permettant un accès direct à une machine.

- **La mémoire projet n'est pas un coffre-fort.**
- **Préférer les fixtures synthétiques** aux données client réelles.
- **Éviter les détails d'ACL/accès sensibles** dans le contexte permanent.
- **Limiter les permissions agent** au strict nécessaire.

18. Professional maturity assessment (0–5)

Dimension	Score	Justification
Requirements engineering	4/5	Structuré, mais gaps de complétude détectés en UAT.
Business analysis	4/5	Règles détaillées ; certaines clarifications sont arrivées tard.
Scope control	4/5	Bon report des zones risquées ; backlog encore large.
Architecture	4/5	Architecture formalisée et séparation Dashboard/Rapports maîtrisée.
Data modeling	4.5/5	Invariants, allocations, minor units et migrations fortement contrôlés.
Security	4/5	RBAC/recovery solides ; verrouillage manuel encore à finaliser.
UX/UI process	3.5/5	Cycles visuels efficaces mais plusieurs défauts ont nécessité des itérations.
Implementation discipline	4/5	Branches/tâches bornées ; supervision encore importante.
Testing	4.5/5	Suites larges, ciblées, négatives et répétées.
Debugging	4.5/5	Cause racine recherchée sur WPF, ACL et recovery.
Change management	4/5	Retours client intégrés ; traçabilité des déviations à renforcer.
Version control	4.5/5	Baselines, SHAs, freezes et merge contrôlé.
Documentation	4/5	Nombreux artefacts ; CDC V1.0 non retrouvé dans le repo.
Release management	3.5/5	Contrôle fort ; RC encore distincte de integration.
AI collaboration	4.5/5	Rôles et sélection de modèles structurés.
Agent supervision	4.5/5	Gates, interdits, preuves et audits présents.
Operational QA	4/5	UAT protégé ; smoke post-intégration à compléter.

Lecture : maturité élevée pour une petite équipe assistée par IA, mais pas encore industrialisée sans réserve. Les principaux écarts concernent complétude, convergence release/integration, traçabilité documentaire et couverture systématique runtime/UI après intégration.

19. Questions / uncertainties

- Le CDC V1.0 validé est-il bien archivé dans le dépôt définitif ?
- Le smoke runtime complet a-t-il été exécuté après le dernier merge sur integration ?
- Quel format d'imprimante ticket est finalement retenu : thermique 58/80 mm ou A4 ?
- Le verrouillage manuel possède-t-il une spécification détaillée complète ?
- Les modifications ventes/achats/dépenses sont-elles définitivement V1.1 ?
- Le modèle de modification d'une opération créée par un autre employé est-il finalisé ?
- Le protocole d'installation/restauration sur nouvelle machine est-il complètement testé ?
- Existe-t-il une stratégie de rétention/rotation des backups ?
- Quelle stratégie de mise à jour post-livraison est prévue ?
- Quelle documentation sera remise au client ?

20. Compact normalized dataset

Normalisation compacte destinée au conditionnement d'un futur agent ; elle exclut les secrets.

FIELD	VALUE
PROJECT_NAME	Logiciel de gestion financière
PROJECT_TYPE	Desktop financial management application
CLIENT_CONTEXT	EL-BARKA EMBALLAGE
OPERATING_MODE	Offline / local Windows
TEAM_MODEL	2 human developers + AI agents
ARCHITECTURE	Modular monolith
TECH_STACK	C#; .NET 10 LTS; WPF; MVVM; EF Core; SQLite; xUnit; local logging; Inno Setup
SOURCE_OF_TRUTH	Validated CDC / functional / data / UX-UI / architecture documents
MONEY_MODEL	long minor units; SQLite INTEGER
TREASURY_RULE	Payments and transfers move treasury; sale/purchase creation does not
OBLIGATION_MODEL	Receivables/payables = obligations minus explicit allocations
ADVANCE_RULE	Overpayment becomes advance; no automatic reallocation
PAYMENT_METHODS	Cash; BankTransfer
ROLES	RESPONSABLE; EMPLOYE
RESPONSIBLE_ONLY	Dashboard; Treasury; Administration; BackupRestore; PermanentDelete; Reports
CLIENT_POLICY	Cross-user read visibility accepted; modification remains controlled
RECOVERY	UI backup/restore; mandatory pre-migration backup; daily local backup
MIGRATIONS	001; 002; 003; 004 only
AI_MODEL	Lead; Codex; Astra; Sol; Luna
PROMPT_PATTERN	Goal; Context; Scope; Constraints; Implementation; Validation; Evidence; Stop
GIT_POLICY	No implementation Git; ACCEPT before commit/push; controlled merge; no force push
QUALITY_GATE	Targeted tests + full suite + build + migrations + drift + security + UAT
CURRENT_INTEGRATION	origin/integration = 7650a487b36389cd0afdfaeae952bd6c9da969aa
CURRENT_RC	origin/release/v1.0-rc = fea6f90da6b9c7e6ada3ef71d68dcc57a2edb4ba
NEXT_PRIORITY	Client UI/features: balances; obligation-centric payments; ticket; manual lock
HIGH_RISK_DEFERRED	Complex financial modifications/deletions requiring stronger invariant design

End state
Le projet fonctionne comme un **système d’ingénierie supervisé par preuves** : l’humain fixe le produit et les gates, les agents exécutent des sous-tâches spécialisées, les audits recherchent les défauts et Git sert de mécanisme de promotion.